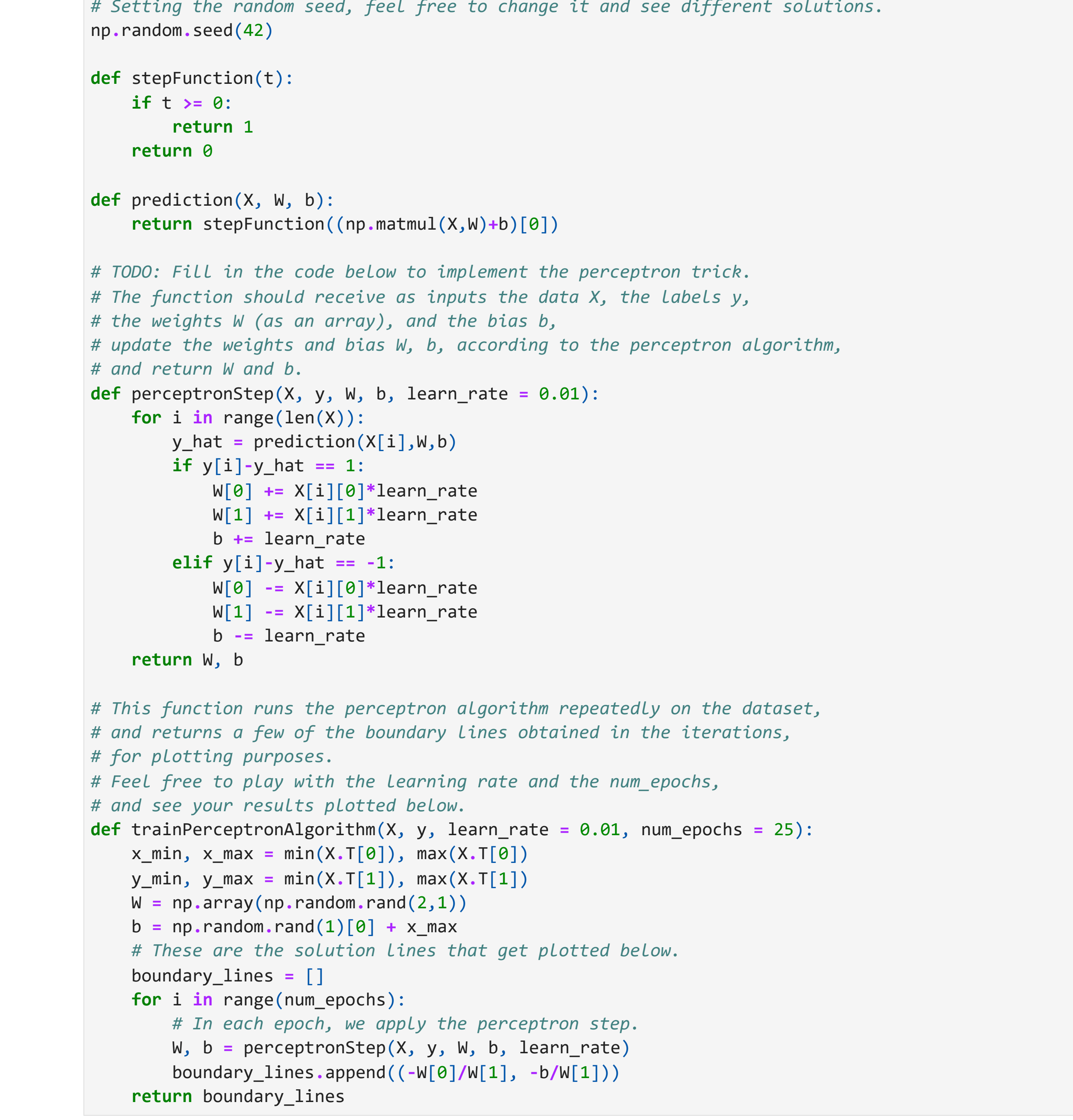
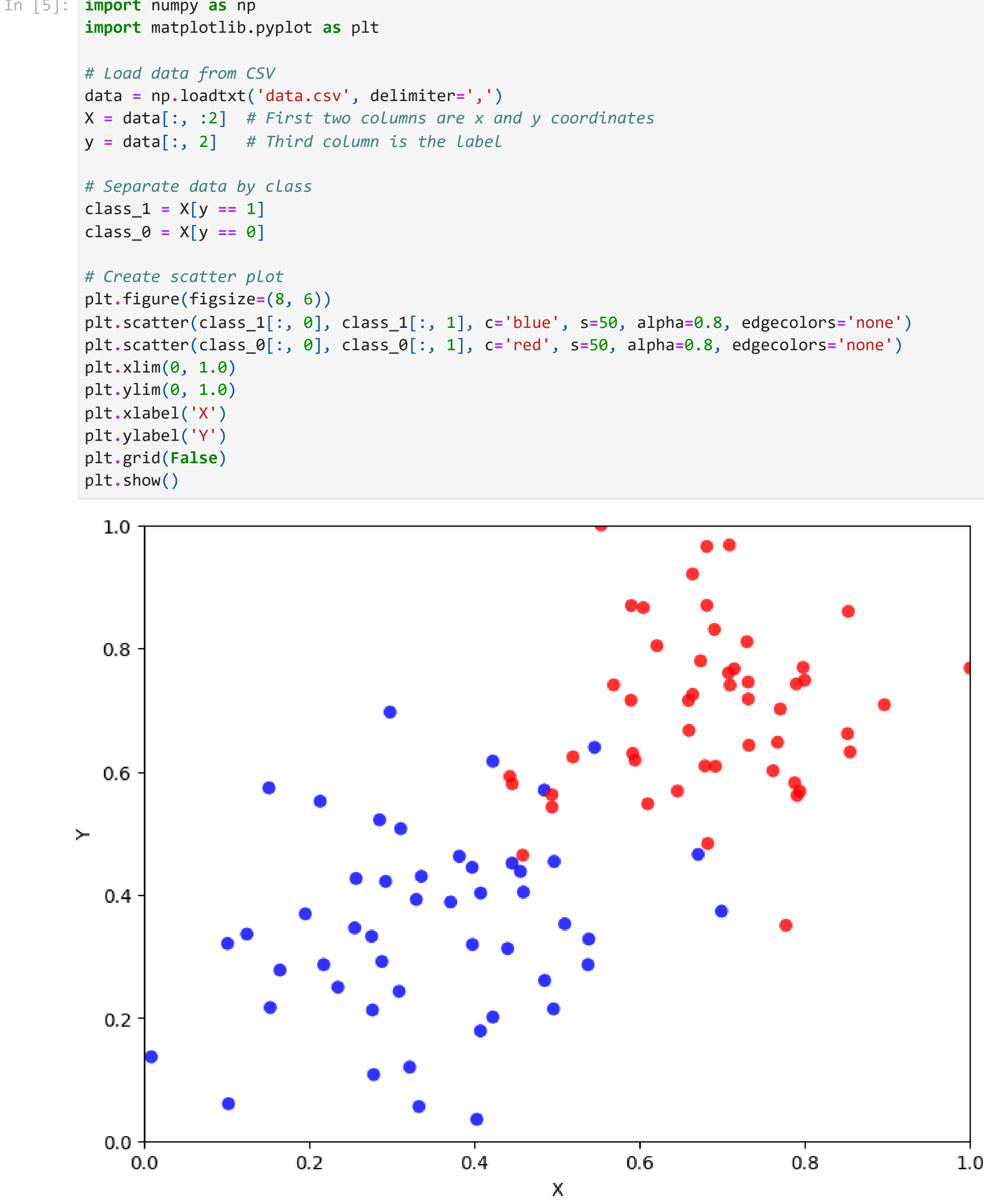


Linear Boundaries



3 or More Labels

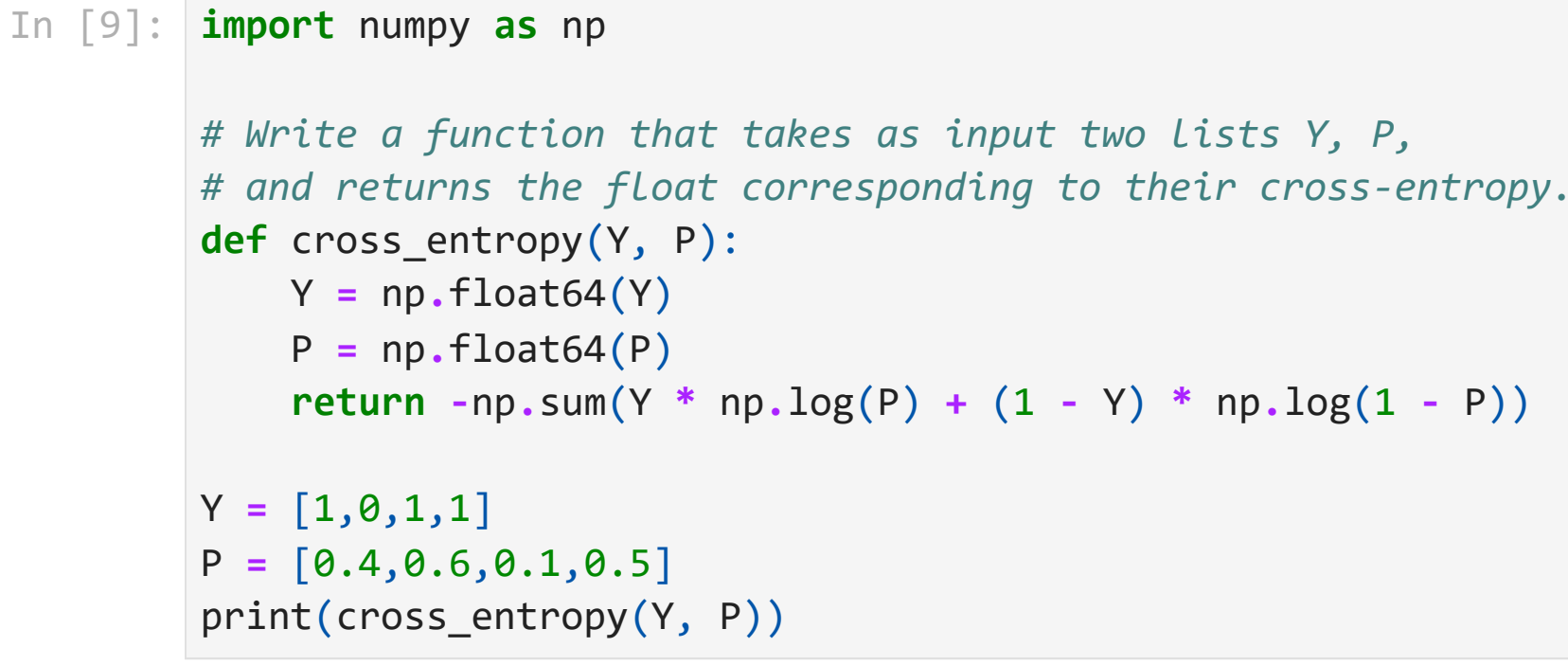
Softmax Algorithm



[np.float64(0.09003057317038046), np.float64(0.24472847105479764), np.float64(0.6652409557748219)]

Cross Entropy

cross entropy for 2 classes



4.828313737302301

cross entropy for 3 or more classes

$$\text{Cross-Entropy} = - \sum_{i=1}^n \sum_{j=1}^m y_{ij} \ln(p_{ij})$$

Linear Regression

$$\text{Error Function} = -\frac{1}{m} \sum_{i=1}^m (1 - y_i) \ln(1 - \hat{y}_i) + y_i \ln(\hat{y}_i)$$

for sigmoid function:

$$E(W, b) = -\frac{1}{m} \sum_{i=1}^m (1 - y_i) \ln(1 - \sigma(Wx^{(i)} + b)) + y_i \ln(\sigma(Wx^{(i)} + b))$$

Gradient Descent

What we do on Gradient Descent

$$\begin{aligned} \hat{y} &= \sigma(Wx + b) \quad \leftarrow \text{Bad} \\ \hat{y} &= \sigma(w_1x_1 + \dots + w_nx_n + b) \\ \nabla E &= \left(\frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n}, \frac{\partial E}{\partial b} \right) \\ \alpha &= 0.1 \quad (\text{learning rate}) \\ w'_i &\leftarrow w_i - \alpha \frac{\partial E}{\partial w_i} \\ b' &\leftarrow b - \alpha \frac{\partial E}{\partial b} \\ \hat{y} &= \sigma(W'x + b') \quad \leftarrow \text{Better} \end{aligned}$$

Gradietn Calculation

The first thing to notice is that the sigmoid function has a really nice derivative. Namely,

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

And now, let's recall that if we have m points labelled $x^{(1)}, x^{(2)}, \dots, x^{(m)}$, the error formula is:

$$E = -\frac{1}{m} \sum_{i=1}^m (y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i))$$

where the prediction is given by $\hat{y}_i = \sigma(Wx^{(i)} + b)$.

Our goal is to calculate the gradient of E , at a point $x = (x_1, \dots, x_n)$, given by the partial derivatives

$$\nabla E = \left(\frac{\partial}{\partial w_1} E, \dots, \frac{\partial}{\partial w_n} E, \frac{\partial}{\partial b} E \right)$$

To simplify our calculations, we'll actually think of the error that each point produces, and calculate the derivative of this error. The total error, then, is the average of the errors at all the points. The error produced by each point is, simply,

$$E = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

In order to calculate the derivative of this error with respect to the weights, we'll first calculate $\frac{\partial}{\partial w_j} \hat{y}$. Recall that $\hat{y} = \sigma(Wx + b)$, so:

$$\frac{\partial}{\partial w_j} \hat{y} = \frac{\partial}{\partial w_j} \sigma(Wx + b) \tag{1}$$

$$= \sigma(Wx + b) (1 - \sigma(Wx + b)) \cdot \frac{\partial}{\partial w_j} (Wx + b) \tag{2}$$

$$= \hat{y} (1 - \hat{y}) \cdot \frac{\partial}{\partial w_j} (Wx + b) \tag{3}$$

$$= \hat{y} (1 - \hat{y}) \cdot \frac{\partial}{\partial w_j} (w_1x_1 + \dots + w_jx_j + \dots + w_nx_n + b) \tag{4}$$

$$= \hat{y} (1 - \hat{y}) \cdot x_j \tag{5}$$

The last equality is because the only term in the sum which is not a constant with respect to w_j is precisely w_jx_j , which clearly has derivative x_j .

Now, we can go ahead and calculate the derivative of the error E at a point x , with respect to the weight w_j .

$$\frac{\partial}{\partial w_j} E = \frac{\partial}{\partial w_j} [-y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})] \tag{6}$$

$$= -y \frac{\partial}{\partial w_j} \log(\hat{y}) - (1 - y) \frac{\partial}{\partial w_j} \log(1 - \hat{y}) \tag{7}$$

$$= -y \cdot \frac{1}{\hat{y}} \cdot \frac{\partial}{\partial w_j} \hat{y} - (1 - y) \cdot \frac{1}{1 - \hat{y}} \cdot \frac{\partial}{\partial w_j} (1 - \hat{y}) \tag{8}$$

$$= -y \cdot \frac{1}{\hat{y}} \cdot \hat{y} (1 - \hat{y}) x_j - (1 - y) \cdot \frac{1}{1 - \hat{y}} \cdot (-1) \hat{y} (1 - \hat{y}) x_j \tag{9}$$

$$= -y (1 - \hat{y}) \cdot x_j + (1 - y) \hat{y} \cdot x_j \tag{10}$$

$$= -(y - \hat{y}) x_j \tag{11}$$

A similar calculation will show us that

$$\frac{\partial}{\partial b} E = -(y - \hat{y})$$

This actually tells us something very important. For a point with coordinates $(x_1, \dots, x_n)$, label y , and prediction $\hat{y}$, the gradient of the error function at that point is $((y - \hat{y})x_1, \dots, (y - \hat{y})x_n, (y - \hat{y}))$. In summary, the gradient is

$$\nabla E = -(y - \hat{y})(x_1, \dots, x_n, 1).$$

If you think about it, this is fascinating. The gradient is actually a scalar times the coordinates of the point! And what is the scalar? Nothing less than a multiple of the difference between the label and the prediction. What significance does this have?

So, a small gradient means we'll change our coordinates by a little bit, and a large gradient means we'll change our coordinates by a lot.

If this sounds anything like the perceptron algorithm, this is no coincidence! We'll see it in a bit.

Gradient Descent Step

Therefore, since the gradient descent step simply consists in subtracting a multiple of the gradient of the error function at every point, then this updates the weights in the following way:

$$w'_i \leftarrow w_i - \alpha [-(y - \hat{y})x_i],$$

which is equivalent to

$$w'_i \leftarrow w_i + \alpha (y - \hat{y})x_i.$$

Similarly, it updates the bias in the following way:

$$b' \leftarrow b + \alpha (y - \hat{y}),$$

Note: Since we've taken the average of the errors, the term we are adding should be $\frac{1}{m} \cdot \alpha$ instead of α , but as α is a constant, then in order to simplify calculations, we'll just take $\frac{1}{m} \cdot \alpha$ to be our learning rate, and abuse the notation by just calling it α .

Gradient Descent Algorithm

Gradient Descent Algorithm:

- Start with random weights: $w_1, \dots, w_n, b$
- For every point $(x_1, \dots, x_n)$:
 - For $i = 1, \dots, n$
 - Update $w'_i \leftarrow w_i - \alpha(\hat{y} - y)x_i$
 - Update $b' \leftarrow b - \alpha(\hat{y} - y)$
- Repeat until error is small